

ФГБОУ ВПО КубГУ

Курс лекций

2026 г

Технологии программирования
5 сем. Язык JAVA

Зав. каф. ВМиИ, к.ф.-м.н,
доцент, Фоменко С.И.

Тема 1:
Введение в язык программирования JAVA

Наиболее востребуемые на рынке труда языки программирования

TIOBE Index for September 2026

https://www.tiobe.com/tiobe-index/

Sep 2026	Sep 2025	Change	Programming Language	Ratings	Change
1	1		Python	17.76%	-8.22%
2	3	↑	C	10.28%	+1.63%
3	2	↓	C++	8.67%	-0.13%
4	4		Java	7.54%	-0.81%
5	5		C#	4.22%	-2.16%
6	6		JavaScript	2.76%	-0.46%
7	7		Visual Basic	2.55%	-0.28%
8	11	↑	SQL	2.16%	+0.29%
9	13	⇈	R	1.69%	+0.27%
10	18	⇈	Rust	1.34%	+0.33%
11	12	↑	Fortran	1.24%	-0.25%
12	8	⇊	Go	1.10%	-1.22%

Наиболее востребуемые на рынке труда языки программирования

TIOBE Index for September 2026

https://www.tiobe.com/tiobe-index/

Very Long Term History

Programming Language	2026	2021	2016	2011	2006	2001	1996	1991	1986
Python	1	3	5	6	8	25	24	-	-
C	2	1	2	2	2	1	1	1	1
C++	3	4	3	3	3	2	2	2	6
Java	4	2	1	1	1	3	15	-	-
C#	5	5	4	5	7	13	-	-	-
JavaScript	6	7	7	10	9	9	20	-	-
Visual Basic	7	6	13	-	-	-	-	-	-
SQL	8	9	-	-	-	37	-	-	-
R	9	11	17	27	-	-	-	-	-
Delphi/Object Pascal	10	19	12	12	10	36	-	-	-
Ada	17	31	28	17	16	18	7	3	2
Lisp	32	33	27	13	13	17	8	6	3
(Visual) Basic	-	-	-	7	5	4	3	5	5

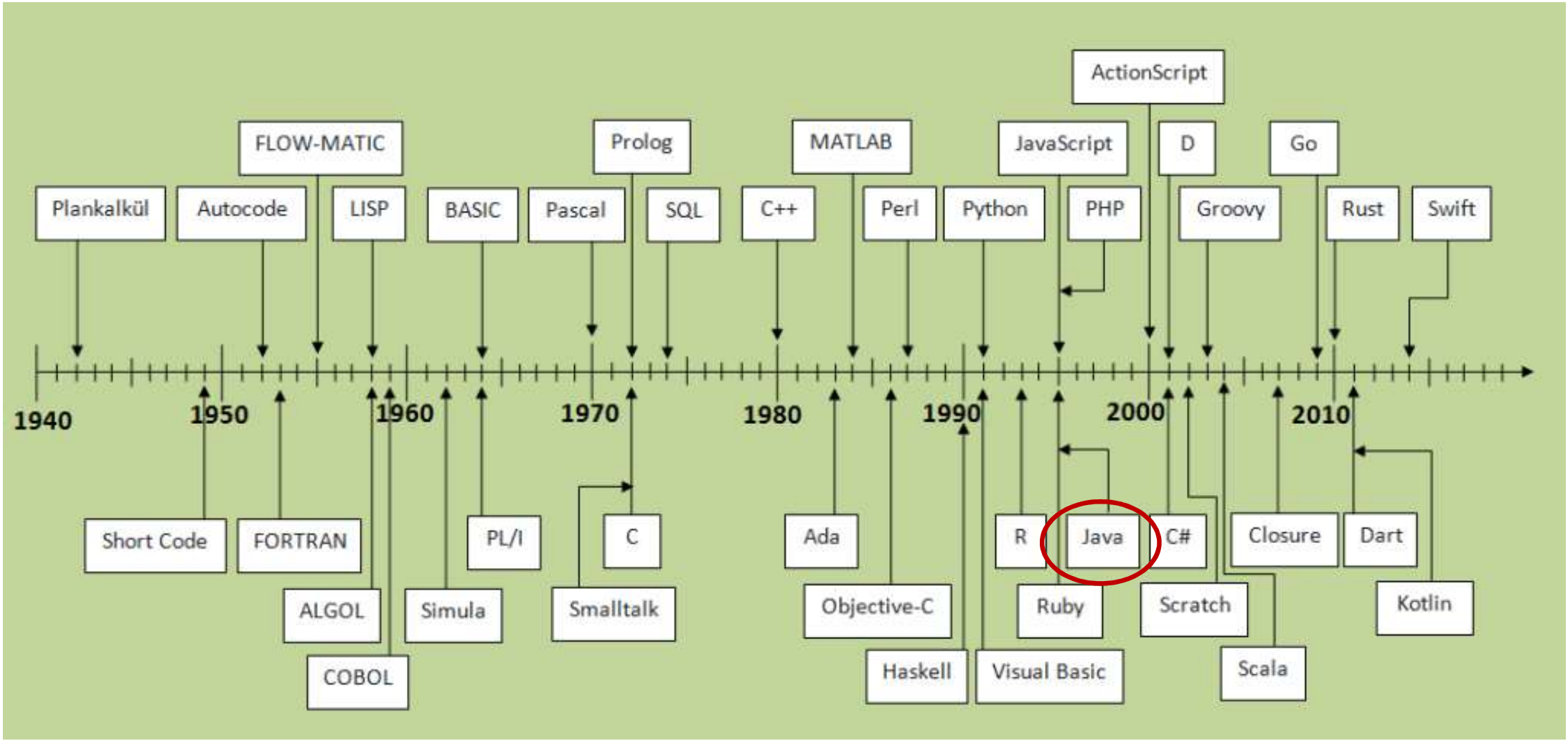


Язык JAVA

JAVA — *статически типизированный, объектно-ориентированный и Си-подобный* язык программирования

Статически типизированный язык означает, что переменные ранее объявленного типа не могут менять свой тип во время исполнения блока команд.

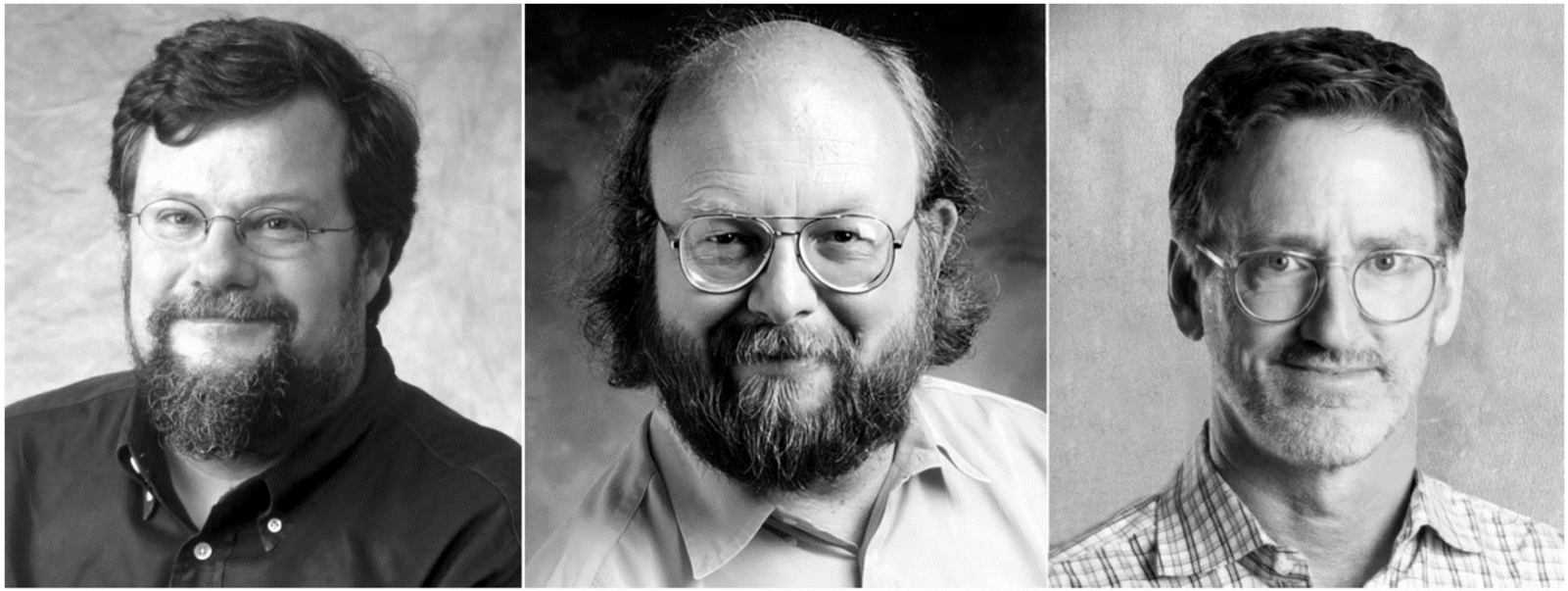
Языки программирования



История

Язык Java разработали Джеймс Гослинг, Патрик Нотон, Крис Уарт, Эд Франк и Майк Шеридан из компании Sun Microsystems, Inc. (позже компанию приобрела Oracle) в 1991 году.

Первоначально язык получил название Oak, но в 1995 году был переименован в Java.





История

Побудительной причиной создания Java стала потребность в независимом от платформы (кросс-платформенном, т.е. архитектурно-нейтральном) языке, который можно было бы использовать для разработки ПО, встраиваемого в разнообразные бытовые электронные устройства, автомобили…

Вторая причина – потребность переносимости программ для Интернета. Она и стала главной движущей силой JAVA.

Интернет помог язык у Java выйти на передний край программирования, а язык Java в свою очередь оказал глубокое влияние на Интернет:

Апплеты Java (на настоящий момент поддержка прекратилась) – маленькие встроенные через HTML программы, работающие «в песочнице»)



Сферы применения

Сфера применения Java очень широка. Вот основные направления:

Корпоративная разработка и Enterprise. Банки, страховые компании, государственные учреждения, логистические и телеком-компании часто выбирают Java для построения масштабных, надёжных и масштабируемых систем: учёт клиентов, обработка транзакций, корпоративные порталы, ERP-системы.

Веб-разработка (бэкенд). На Java пишут серверную логику для интернет-магазинов, банковских порталов, онлайн-сервисов (стриминг, бронирование). Фреймворки вроде Spring Boot стали стандартом для создания микросервисов.

Мобильная разработка (Android). Долгое время Java была основным языком для Android. Сейчас активно используется Kotlin, но огромная база существующих приложений написана на Java, поэтому специалисты по ней по-прежнему востребованы.

Облачные и распределённые системы. Java подходит для построения инфраструктур в облаке (AWS, Google Cloud и др.), сервисов потоковой аналитики.



Сферы применения

Сфера применения Java очень широка. Вот основные направления:

….

Обработка больших данных. Ядро таких платформ, как Hadoop, Apache Spark, написано на Java — это позволяет строить системы для анализа терабайтов информации в реальном времени.

Встраиваемые системы и IoT. Java применяют в банкоматах, POS-терминалах, смарт-гаджетах, промышленном оборудовании.

Наука, образование, разработка инструментов. На Java создают исследовательские инструменты, среды разработки (например, IntelliJ IDEA, Eclipse, NetBeans), системы управления проектами (Jira).



Различия между Java, C и C++

Совместимость: По большей части C++ является надстройкой над Си, поэтому большая часть кода на Си будет компилироваться и в C++, но в обратную сторону это не работает.

Язык Java очень похож на C++, но это все-таки другой язык программирования. Языки Java и C++ не совместимы.



Различия между Java, C и C++

Управление динамической памятью

C: полностью ручное управление через malloc/free. Это даёт максимальный контроль, но и максимальную ответственность: легко получить утечки или ошибки доступа к памяти. Для численных расчётов это критично: нужно вручную следить за выделением больших массивов и их освобождением.

C++: ручное управление через new/delete, но с современными средствами (умные указатели std::unique_ptr, std::shared_ptr) можно сделать управление безопасным и предсказуемым.

Java: автоматическое управление памятью через сборщик мусора (GC). Это сильно снижает риск ошибок, но делает поведение менее предсказуемым: возможны паузы на сборку мусора, что нежелательно в расчётах реального времени или при жёстких требованиях к задержкам.



Различия между Java, C и C++

Производительность

C: максимально близок к аппаратуре, минимальные накладные расходы. Идеален для низкоуровневых оптимизаций, работы с SIMD, векторизации, прямого доступа к памяти. Часто используется в высокопроизводительных численных библиотеках.

C++: почти такая же производительность, как у C, плюс возможности объектно-ориентированного и обобщённого программирования (шаблоны, перегрузки). Многие инженерные библиотеки (например, для МКЭ) написаны на C++.

Java: благодаря JIT-компиляции и оптимизациям JVM производительность близка к C++ для многих задач, но есть накладные расходы: GC-паузы, проверки границ массивов, косвенность при вызове методов. Для тяжёлых численных расчётов (большие матрицы, итерационные методы) Java может уступать C/C++.



Различия между Java, C и C++

Платформенная независимость

C/C++: компилируются в машинный код под конкретную платформу. Для кроссплатформенности нужно отдельно собирать под каждую ОС и архитектуру.

Java: компилируется в байт-код, который выполняется на любой JVM. Принцип «Write Once, Run Anywhere» сильно упрощает развёртывание на разных системах, что удобно для распределённых вычислений.



Различия между Java, C и C++

Реализованные парадигмы программирования

C: процедурный язык, минимум абстракций. Подходит для низкоуровневых задач, но для сложных моделей код может стать громоздким.

C++: многопарадигменный язык (процедурный, ООП, обобщённое программирование, функциональное). Шаблоны и метапрограммирование позволяют писать очень гибкие и эффективные библиотеки для численных методов.

Java: строго объектно-ориентированный язык с поддержкой функциональных элементов (лямбды, Stream API). Удобен для построения больших, поддерживаемых систем, но менее гибок в низкоуровневых оптимизациях.



Различия между Java, C и C++

Безопасность и надёжность

C: небезопасен по умолчанию: указатели, арифметика указателей, отсутствие проверок границ массивов. Ошибки легко приводят к неопределённому поведению.

C++: более безопасен при использовании современных практик (умные указатели, контейнеры STL, запрет на «сырые» указатели), но всё ещё допускает небезопасные операции.

Java: безопасные абстракции: нет указателей, проверки границ массивов, автоматическое управление памятью. Это снижает количество ошибок, но не устраняет их полностью.



Различия между Java, C и C++

Сравнение создания главной функции (точки входа) программы

C/C++: функция **main** – точка входа.

```c
#include <stdio.h>
int main(void) {
    …
    return 0;
}
```

```c
#include <iostream>
int main() {
    …
    return 0;
}
```

Java: Программа – объект определённого класса (имя программы), содержащего метод **main** с префиксами `public static`

```java
import java.util.Scanner;
public class MyProgram {
    public static void main(String[] args)
    {
        …
    }
}
```



Различия между Java, C и C++

Сравнение консольного ввода-вывода

C: используются функции из `<stdio.h>`: `printf`, `scanf`, `fprintf`, `fscanf`.

```
double x;
scanf("%lf", &x);
printf("x = %.6f\n", x);
```

C++: потоковый ввод-вывод из `<iostream>`: `std::cin`, `std::cout`, операторы `>>` и `<<`.

```
double x;
std::cin >> x;
std::cout << "x = " << std::fixed <<
std::setprecision(6) << x << "\n";
```

Java: Объект System.out (для вывода) и класс Scanner (для ввода) (подключение: import java.util.Scanner;)

```
Scanner input = new Scanner(System.in);
double x = input.nextDouble();
System.out.printf("x = %.6f%n", x);
```



Различия между Java, C и C++

Java: Объект System.out (для вывода) и класс Scanner (для ввода) (подключение: import java.util.Scanner;)

```
import java.util.Scanner;
public class Program {
    public static void main(String[] args)
    {
        System.out.println("Hello, I’m Java!");
        System.out.print("How are you? Input your name:");

        Scanner in = new Scanner(System.in);
        String name = in.nextLine();
        System.out.printf("%nInput your age:");
        int a=in.nextInt();
        System.out.printf("%Hello, %s! You are %d years
            old.%n", name, age);
    }
}
```

Операторные скобки

```
{
   что-то
}
```

Примеры:

```
int main()
{
   тело программы
}
```

```
if(2>1)
{
 значит что-то делаем
}
```

Разделение операторов: точка с запятой

```
оператор1 ; оператор2 ;
```

Пример:

```
a=2;
b=a+1;
```



Комментарии

```
// однострочный комментарий
```

```
/* многострочный
   комментарий
*/
```

Комментарии

Примеры:

```
/* программа вычисления суммы
   двух чисел
*/
int main()
{
  int a,b,c; // объявления переменных

  a=1;     // присвоение значения переменной a
  b=2;     // присвоение значения переменной b
  c=a+b; // вычисление суммы
  return 0; // выход из программы без ошибок
}
```



Основы: импорт классов

Ранее созданные (встроенные) классы можно подключить с помощью.

Import. Для этого надо знать полное (квалифицированное) имя класса, т.е.еще путь (пакет), где он расположен

```
import пакет1.пакет2.пакет3.Класс
```

пакет1.пакет2.пакет3 – это иерархия пакетов. (пакет1 – верхний уровень, пакет3 – пакет третьего уровня (вложенный в пакет2)

Класс – имя подключаемого класса в пакете 3.

Фактически в файловой системе путь следующий:

/пакет1/пакет2/пакет3/Класс.class

(Класс.class – байт код Класс.java)



Объявление переменных и присваивание

Объявление переменных определенного типа

```
[спецификатор] тип_данных  имя_переменной[=
значение][, имя[=значение]]
```

Присваивание

```
имя_переменной = выражение_или_значение
```

```
public class Main {
  public static void main(String[] args)
  {
        int a;
        a = 5;
        double b = 2.5;
        double c = a*b;
        System.out.println(c);
 }
}
```



Объявление переменных и присваивание

Автоматическое определенного типа

```
var  имя_переменной = значение_выражение
```

Тип данных значения должен быть однозначно определён

```
public class Main {
  public static void main(String[] args)
  {
        int a= 5;
        var b = 2.5;  // double
        var c = a*b; // double
        System.out.println(c);
 }
}
```



Имя переменной

1. В именах разрешено использовать только алфавитных символов, цифр и символа подчеркивания (_).

2. Символы в верхнем и нижнем регистре рассматриваются как разные.

3. Первым символом имени не должна быть цифра.

4. Первым символом имени не рекомендуется использовать символ подчеркивания (_).

5. В качестве имени нельзя использовать ключевое слово C++.

```
int po3; // допустимое
int Po3; // допустимое и отличающееся от po3
int PO3; // допустимое и отличающееся от двух предыдущих
int 4ever;// недопустимое, потому что начинается с цифры
int double;// недопустимое:double является ключевым словом C++
int begin; // допустимое:begin является ключевым словом Pascal
```

Встроенные типы данных

	Тип	Диапазон значений	Размер (байт)
Логический тип →	boolean	true и false	1
Целочисленные типы	byte	-128 … 127	1
	short	-32 768… 32 767	2
	int	-2 147 483 648 … 2 147 483 647	4
	long	−9 223 372 036 854 775 808 … +9 223 372 036 854 775	8
Типы с плавающей точкой	float	3.4e-38 … 3.4e+38	4
	double	1.7e-308 … 1.7C+308	8
Символьный тип	char	0 … 65 535	2
	String	Длина макс 2 147 483 647 символов	



Операторы

Арифметические операторы

Оператор	Описание	Пример
+	Сумма двух операндов	2 + 1 даст 3
-	Вычитание	2 – 1 даст 1
*	Умножение	2 * 3 даст 6
/	Деление (зависит от типа)	4 / 2 даст 2
%	Деление по модулю	3%2 даст 1
++	Инкремент, увеличивает значение на единицу	3++ даст 4
– –	Декремент, уменьшает значение на единицу	3– – даст 2



Операторы

Операторы сравнения: результат bool (true, false)

Оператор	Описание	Пример
==	Равенство между операндами	(1 == 2) даст false.
!=	Неравенство межу операндами	(1! = 2) даст true.
>	Операнд 1 больше операнда 2.	(1> 2) даст false.
<	Операнд 1 меньше операнда 2.	(1 <2) даст true.
> =	Операнд 1 больше или равен операнду 2.	(1> = 2) даст false.
<=	Операнд 1 меньше или равен операнду 2.	(1 <= 2) даст true.



Операторы

Логические операторы: результат bool (true, false)

Оператор	Описание	Пример
&&	Логическое И	(true && false) даст false.
||	Логическое ИЛИ	(true || false) даст true.
!	Логическое отрицание	! (true && false) даст true.

Побитовые операторы

Оператор	Описание	Пример
&	Побитовое И	(2#101 & 2#011) даст 2#001 = 1
|	Побитовое Или	(2#101 | 2#011) даст 2#111 = 7
^	Исключающее Или (XOR)	(2#101 ^ 2#011) даст 2#110 = 6
~	Бинарное отрицание	(~ 2#101) даст 2#010 = 2
<<	Битовый сдвиг влево первого операнда на количество бит, заданного вторым операндом	2#0101 << 1 даст 2#1010 = 10
>>	Битовый сдвиг вправо первого операнда на количество бит, заданного вторым операндом	2#0101 >> 1 даст 2#-0010 = 2

Операторы присваивания

Оператор	Описание	Пример
=	Простой оператор присваивания, присваивает значения из правого операнда левому операнду	C = A + B присваивает значение A + B в C
Бинарный_операнд =	Выполняет бинарный операнд с левым и правым операндом и присваивает значение левому операнду	C + = A эквивалентно C = C + A C –=A ⇔ C = C – A C*=A ⇔ C = C*A

Другие операторы

Оператор	Описание	Пример
условие ? X : Y	Если Условие истинно, то возвращается X, иначе Y.	(1>2) ? 4 : 5 даст 5
(тип)	Преобразование (явное приведение) типа	int a; a=(int) 2.2; // a=2



Операторы

Другие операторы

Оператор	Описание	Пример
new	Выделение памяти и присвоение адреса указателю p = new тип или p = new тип(инициализатор) p = new тип[size] -- одномерный массив	double *a a = new double[10] int b=new int int c=new int(1) // c =1, b=0, a -- массив
delete	Освобождение ранее выделенной памяти оператором new	delete[]a; delete b; delete c;
sizeof	Возвращает размер переменной.	sizeof(2.0) даст 4 sizeof('a') даст 1



Условные операторы

Условный оператор if .. else

```
if (bool_условие) один_оператор;
```

```
if (bool_условие)
{
 // блок операторов , если верно
}
```

```
if (bool_условие)
{
  // блок операторов, если верно
}
else
{
 // блок операторов, если неверно
}
```

Условный оператор if .. else

Примеры:

```
float x=1,y=2;

if (x<y)
{
    System.out.println("x<y");
}
else
{
    System.out.println("x>=y");
}
```

Оператор выбора switch

```
switch (выражение)
{
    case val_1:
         тело1;  // операторные скобки не нужны
         break; // выход из switch
    case val_2:
         тело2;
         break; // выход из switch
    …
    case val_3:
           тело3;
    case val_4:
          тело4; // для случая val_3 также выполняется!
         break; // выход из switch
    default:
         действия по умолчанию; // необязательно
}
```

Оператор цикла с параметрами (for)

```
for(инициализация; условие; итерируемое выражение
после основного тела цикла)
{
  тело цикла
}
```

Примеры:

```
float S=0;
int n=10;

for(int i=1; i <= n; ++i)
{
  S+=1.0/(i*i); // 1/(i*i) – неверно. Почему?
}
System.out.printf("Sum of %d item is %f",n,S);
```

Оператор цикла с предусловием (while)

```
while(условие)
{
   тело цикла, если условие выполняется
}
```

Примеры:

```
float S=0;
int n=10, i=0;

while(i<n)
{
   i++;
   S+=1.0/(i*i);
}
```

Оператор цикла с постусловием (do …while)

```
do
{
   тело цикла, если условие продолжает выполняться
}
while(условие)
```

Примеры:

```
float S=0;
int n=10, i=0;

do
{
   i++;
   S+=1.0/(i*i);
}
while(i<n);
```



Массивы

Массивы в Java являются динамическими типами данных

Объявление (2 эквивалентных варианта)

```
тип      имя_переменной[]
тип[]    имя_переменной
```

Для создания используется оператор **new**:

```
имя_переменной = new тип[размер]
```

```
int[] A=new int[10]
```

Возможна инициализация во время объявления (список иниц.):

```
тип[]   имя_переменной={value1, value2, …}
```

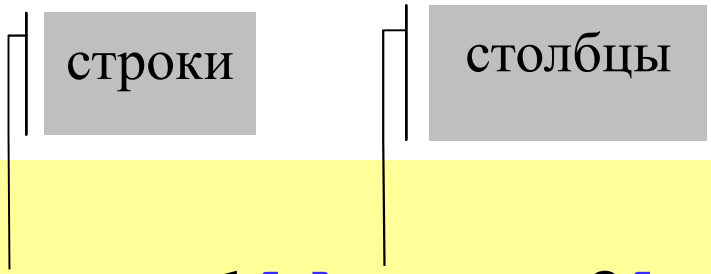
```
int[] A={1,2,3,4,5,6,7,8,9,10}
```

Обращение к элементам массива

```
имя_переменной[index],      0<=index<размер
```

```
A[0]=0; System.out.print(A[0]);
```

Многомерные массивы




```
тип[][]  имя_переменной
имя_переменной = new тип[размер1][размер2]
```

Обращение к элементам

```
имя_переменной[index1][index2]
0<= index1 < размер1, 0<= index2 < размер2,
```

```
int[][] nums=new int[3][5];

for(int i=0; i<3; i++)
  for (int j=0; j<5; j++)
        nums[i][j]=i*i-j*j;

 System.out.printf("%n%d",nums[0][4]);
```



Операторы for (второй вариант)

Оператор цикла с параметром типа for-each:

```
for(тип параметр : коллекция)
{
  тело цикла
}
```

Примеры:

```
int [ ] nums = { 1 , 2 , 3 , 4 , 5} ;
int sum = 0 ;
for ( int x : nums ) sum += x*x;
System.out.println("Sum is " + sum);
```



Литература

1. Г. Шилдт, Java. Полное руководство, 12-е изд. : Пер. с англ. - СПб. "Диалектика"; 2023. - 1344 с.